

52. defiler(), et l'octet qui boucle

La carte fait 256 pixels — la même taille qu'un octet.

1. Ce qu'il faut comprendre

`defiler(x, y)` fait glisser tout le fond. La carte fait **256 pixels de côté** et revient toute seule à zéro.

`glisse` est un octet : il boucle exactement au bon moment, **sans qu'on ait rien à écrire**. C'est une coïncidence heureuse du matériel, et l'un des rares endroits où le débordement d'un octet est ce qu'on veut.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **301 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	92 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	16 o	← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 1	13 o sur place, dans main() + AdresseCase (10 o)
bouton()	× 2	10 o sur place, dans main() + LireManette (37 o)
defiler()	× 1	7 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

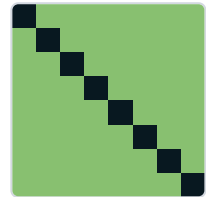
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : remplit l'écran d'un motif en diagonale ; GAUCHE / DROITE
// font glisser tout le décor, pixel par pixel.
//
// defiler(x, y) déplace la « fenêtre » par laquelle on voit le fond, en pixels.
// Rien n'est redessiné : c'est une seule écriture, très rapide.
// La carte du fond fait 256 pixels de large : « glisse », un uint8_t (0 à 255),
// repasse de 255 à 0 juste au moment où la carte recommence. Elle boucle.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <poser>      // pose une tuile sur une case du fond
#include <bouton>     // lit un bouton de la manette
#include <defiler>    // fait glisser tout le fond
```

2. UN DESSIN — MOTIF

```
Tuile MOTIF = {
    // Une diagonale de # sur un fond de -.
    "#-----", "-#-----", "--#-----", "---#-----",
    "----#-----", "-----#-", "-----#-", "-----#",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

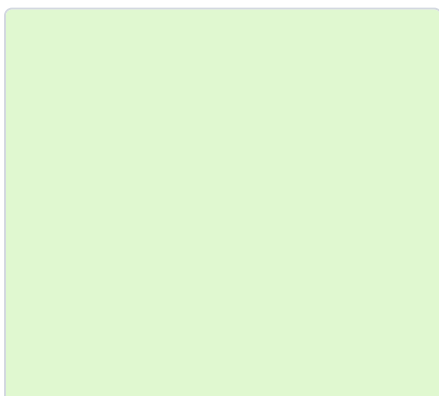
```
int main() {
    // Le jeu commence ici.
    for (uint8_t y = 0; y < 18; y++) // Chaque ligne y (0 à 17)...
        for (uint8_t x = 0; x < 20; x++) // ... chaque colonne x (0 à 19) :
            poser(x, y, MOTIF);        // le motif. (Sans accolades, chaque for
                                        // ne répète que l'instruction qui le suit.)

    uint8_t glisse = 0;                // De combien de pixels le décor a glissé.

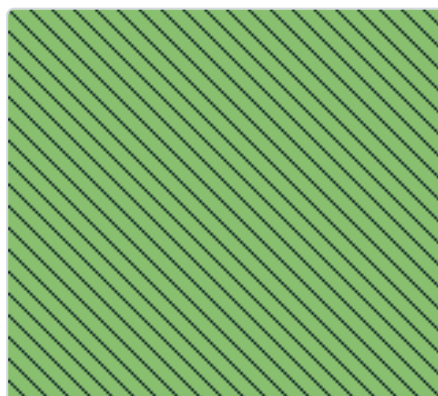
    while (true) {                    // Répète pour toujours :
        if (bouton(DROITE)) glisse++; // DROITE : un pixel de plus
        if (bouton(GAUCHE)) glisse--; // GAUCHE : un pixel de moins (0 - 1 donne 255)
        defiler(glisse, 0);           // On déplace la fenêtre.
        image();                      // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



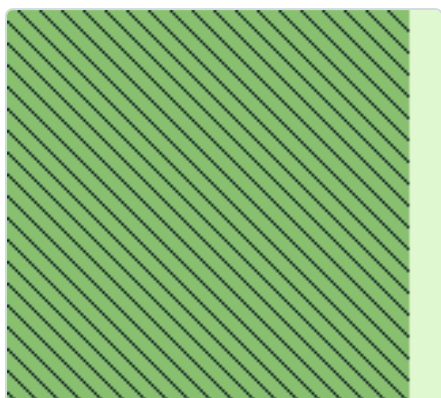
la toute première image — la console vient de démarrer — image 1



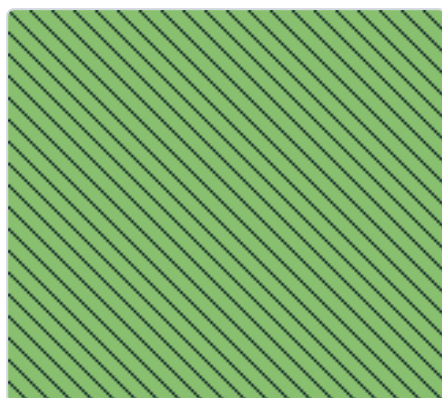
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

DROITE et GAUCHE font glisser le motif sans fin.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ DROITE fait défiler (0 → 30)
- ✓ le défilement reste dans un octet
- ✓ il repasse par zéro sans rien casser (29)
- ✓ le motif couvre tout l'écran

Ensuite : **52.1. Chapitre 8 — Le son : ce qui vient** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++